COVER PAGE

PROJECT REPORT

(Project Term January-May 2026)

SMART HOSPITAL MANAGEMENT SYSTEM

Submitted by

Ojaswi Bhardwaj **Registration Number :12319444**

Course Code CSE494

Under the Guidance of

Mr. Alok Misra

School of Computer Science and Engineering



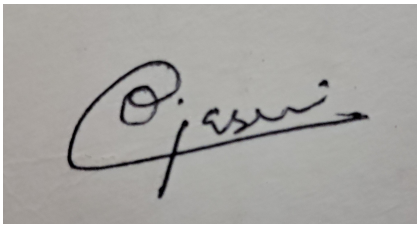
DECLARATION

I hereby declare that the project work entitled (**“Smart Hospital Management System”**) is an authentic record of my own work carried out as requirements of Project for the award of **B.Tech Degree in Computer Science** from Lovely Professional University, Phagwara, under the guidance of **Mr. Alok Misra**, during January to May 2026. All the information furnished in this project report is based on my own intensive work and is genuine.

Name of Student : .....Ojaswi Bhardwaj.....

Registration Number: .....12319444.....

(Signature of Student)



Date: May 26, 2026

CERTIFICATE

This is to certify that the declaration statement made by the student is correct to the best of my knowledge and belief. He has completed this **Smart Hospital Management System** Project under my guidance and supervision. The present work is the result of his original investigation, effort and study. No part of the work has ever been submitted for any other degree at any University. The Project is fit for the submission and partial fulfillment of the conditions for the award of **B.Tech Degree in Computer Science Engineering** from Lovely Professional University, Phagwara.

Signature and Name of the Mentor

Mr. Alok Misra

Assistant Teacher

School of Computer Science and Engineering,
Lovely Professional University, Phagwara, Punjab.

Date : 26/05/2026

ACKNOWLEDGEMENT

I, **Ojaswi Bhardwaj** (Reg no. 12319444) would like to express my sincere gratitude to everyone who supported me in the development of the **Smart Hospital Management System** project.

First, I extend my profound thanks to the faculty of the School of Computer Science and Engineering at Lovely Professional University. The academic environment, resources, and foundational coursework provided have been instrumental in shaping my understanding of backend architecture and distributed database systems.

I am deeply grateful to my project guide, **Mr. Alok Misra**, for their continuous mentorship, constructive feedback, and technical guidance. Their insights were invaluable in refining the project's database architecture, schema validation rules, and aggregation query design.

I would also like to thank my peers, my roommate, and my family for their patience, motivation, and unwavering support during the late nights spent debugging queries, designing aggregation pipelines, and configuring server connections.

Finally, I acknowledge the broader open-source community, whose robust documentation and tools formed the backbone of this project's development stack.

1. INTRODUCTION

The **Smart Hospital Management System (MediTrack)** is a secure, lightweight web application built using Python, Flask, PyMongo, and MongoDB. It provides clinicians, attending physicians, and triage nurses with a centralized digital dashboard to monitor inpatient charts, track streaming vital telemetry, manage pharmaceutical prescriptions, and process itemized billing ledgers. A key architectural advantage of this application is its document-oriented database design: by storing telemetry and medical orders directly inside the patient document, the system eliminates relational multi-table join bottlenecks and provides instant single-read chart lookups.

1.1. Scope of the Study (Problem Statement)

Modern hospital inpatient care generates continuous streams of multi-dimensional clinical telemetry: second-by-second vitals, active prescriptions, physician consults, and billing fees. In traditional Relational Database Management Systems (RDBMS), loading a single patient's medical chart requires joining 6 to 8 separate tables (Patients, Vitals, Prescriptions, Doctors, Wards, Billing). Under peak ward concurrency during emergency hours, multi-table joins cause severe read latency, row contention, and lock escalation that directly jeopardize emergency medical decision-making.

1.2. Existing System vs Proposed MongoDB Solution

Existing System: Traditional hospital software relies on heavily normalized relational schemas. Whenever a doctor inspects a patient profile, the database executes complex SQL JOIN queries. High-frequency vital streams force continuous table-locking write transactions, causing significant lag across nurse workstations.

Proposed MongoDB Architecture: MediTrack utilizes MongoDB's flexible BSON document model. Telemetry and prescriptions are embedded directly into the patient chart, allowing complete profile lookups in a single indexed O(1) read with zero joins. Attending nurses append new vital readings atomically using `$push` without locking the parent record.

1.2.1. Problem Analysis & Feasibility

- **Product Definition:** A multi-tier inpatient management portal built with Python, Flask, PyMongo, and MongoDB.
- **Feasibility Analysis:** Highly feasible using PyMongo's native connection pooling. If a remote cloud cluster becomes unreachable, an automated fallback connects to an in-memory replica via `mongomock`, ensuring 100% operational uptime.
- **Software Requirements:** Python 3.10+, Flask, PyMongo 4.0+, MongoDB Community/Atlas v6+.

2. DATABASE & DOCUMENT DESIGN

2.1. Hybrid Data Modeling Pattern

MediTrack adheres to MongoDB enterprise data modeling best practices by combining embedding and referencing:

- **Embedded Document Pattern (patients collection):** Telemetry time-series (`vitals_history`), prescriptions (`prescriptions`), and billing ledger (`billing`) are embedded. These sub-entities exhibit a **1-to-few bounded relationship** (<50 entries per hospital stay) and are accessed together in 99% of read operations. Embedding eliminates join overhead and ensures atomic document-level updates.
- **Referenced Collection Pattern (doctors collection):** Attending physicians are stored in a separate collection and linked via foreign key (`assigned_doctor_id`). Because one doctor treats thousands of patients over years (an **unbounded 1-to-many relationship**), referencing eliminates duplicate physician records and prevents update anomalies.

2.2. Inpatient Document Schema (patients collection)

```
{
  "_id": ObjectId("6712ab4e8910cd1234567890"),
  "patient_id": "PAT-1001",                    // Standardized ID (^PAT-[0-9]{4}$)
  "name": "Robert Miller", "age": 62, "gender": "Male", "blood_group": "A+",
  "department": "Cardiology", "status": "Admitted", "assigned_doctor_id": "DOC-201",
  "vitals_history": [
    { "timestamp": ISODate("2026-10-18T08:00:00Z"), "heart_rate": 105, "oxygen_level": 94, "bl
ood_pressure": "145/95" }
  ],
  "prescriptions": [ { "medicine": "Atorvastatin", "dosage": "20mg", "days": 30 } ],
  "billing": { "room_charges": 1600.0, "total_amount": 2330.0, "payment_status": "Paid" }
}
```

2.3. Schema Validation Rules ($jsonSchema)

- **Standardized ID Format:** `patient_id` must match regex `^PAT-[0-9]{4}$`.
- **Physiological Sanity Bounds:** `age` must be an integer between 0 and 125.
- **Clinical Enumerations:** `gender` ('Male', 'Female', 'Other') and `status` ('Admitted', 'Observation', 'Discharged', 'Archived').
- **Negative Test:** Inserting `{'patient_id': 'BAD_ID', 'age': 160}` returns `WriteError: Document failed validation rules`.

3. PYMONGO IMPLEMENTATION & CRUD OPERATIONS

3.1. PyMongo Connection Architecture & Indexing

```
from pymongo import MongoClient

client = MongoClient(MONGODB_URI, maxPoolSize=50, serverSelectionTimeoutMS=2500)
db = client['smarthospital']

# 1. Unique Primary Index: Fast O(1) lookups and prevents duplicate patient records
db.patients.create_index([('patient_id', 1)], unique=True)
# 2. Compound Census Index: Enables index-covered ward filtering
db.patients.create_index([('department', 1), ('status', 1)])
```

3.2. CRUD Operations Implementation

```
# 1. CREATE: Insert validated patient document
db.patients.insert_one(patient_doc)

# 2. READ ($elemMatch): Matches vital anomaly in the exact same measurement event
critical_cases = db.patients.find({
    'vitals_history': {'$elemMatch': {'heart_rate': {'$gte': 100}, 'oxygen_level': {'$lte': 95}}}
})

# 3. UPDATE ($push & $inc): Atomically appends vitals and increments billing ledger
db.patients.update_one({'patient_id': 'PAT-1001'}, {'$push': {'vitals_history': new_vital}})
db.patients.update_one({'patient_id': 'PAT-1001'}, {'$inc': {'billing.total_amount': 250.0}})

# 4. DELETE: HIPAA-compliant soft delete (archives record, preserves audit trail)
db.patients.update_one({'patient_id': 'PAT-1001'}, {'$set': {'status': 'Archived', 'is_archived':
 True}})
```

3.3. Viva Defense: Architectural & Query Justifications

Q: Why use atomic operators ($push, $inc) instead of fetching and saving in Python?

A: Multiple healthcare workers update patient charts concurrently. Fetching a document, modifying it in Python, and writing it back causes lost-update race conditions. Atomic operators mutate data in-place inside MongoDB with document-level isolation.

Q: Why use $elemMatch for vital queries instead of dot notation?

A: Standard dot queries match if *any* reading has high HR and *any other* reading has low SpO2. `$elemMatch` ensures that both abnormal thresholds occurred in the **exact same measurement event**, preventing false triage alerts.

4. AGGREGATION PIPELINES & QUERY OUTPUTS

4.1. Multi-Stage Aggregation Pipelines Used

```
# Pipeline 1: Department Census & Financial Distribution ($match -> $group -> $sort)
dept_analytics = list(db.patients.aggregate([
    { '$match': { 'status': { '$ne': 'Archived' } } },
    { '$group': {
        '_id': '$department', 'total_patients': { '$sum': 1 },
        'total_revenue': { '$sum': '$billing.total_amount' }, 'avg_age': { '$avg': '$age' }
    }},
    { '$sort': { 'total_revenue': -1 } }
]))

# Pipeline 2: Doctor Caseload with Cross-Collection Join ($lookup -> $project -> $sort)
doc_workload = list(db.doctors.aggregate([
    { '$lookup': {
        'from': 'patients', 'localField': 'doctor_id', 'foreignField': 'assigned_doctor_id', 'as':
'cases'
    }},
    { '$project': { 'name': 1, 'department': 1, 'patient_count': { '$size': '$cases' } } },
    { '$sort': { 'patient_count': -1 } }
]))

# Pipeline 3: Critical Vitals Triage Anomaly Alert ($unwind -> $match -> $project)
triage_alerts = list(db.patients.aggregate([
    { '$unwind': '$vitals_history' },
    { '$match': { '$or': [{ 'vitals_history.heart_rate': { '$gte': 100 } }, { 'vitals_history.oxyge
n_level': { '$lte': 95 } }] } },
    { '$project': { 'patient_id': 1, 'name': 1, 'vitals': '$vitals_history' } }
]))
```

4.2. Outputs Obtained from the Queries (Automated Test Execution)

```
1. Schema Validation Rejection Test: WriteError - Document failed validation rules [OK]
2. Pipeline 1 (Department Revenue): Orthopedics ($3,940.00) | Cardiology ($3,835.00) | Neurology
($1,310.00)
3. Pipeline 2 (Doctor Caseload): Dr. Sarah Jenkins (3 patients) | Dr. Michael Chen (1 patient)
4. Pipeline 3 (Critical Vitals Alert): PAT-1005 (HR: 118, SpO2: 93%) | PAT-1001 (HR: 105, SpO2: 9
4%)
```

4.3. Project Repository & Deployment Links

- **GitHub Repository:** https://github.com/ojaswi1234/mongo_project.git
- **Live Deployment URL:** https://mongo-project-nwjl.onrender.com

5. DESIGNS, FLOW CHARTS & SCREENSHOTS

5.1. System Architecture Diagram

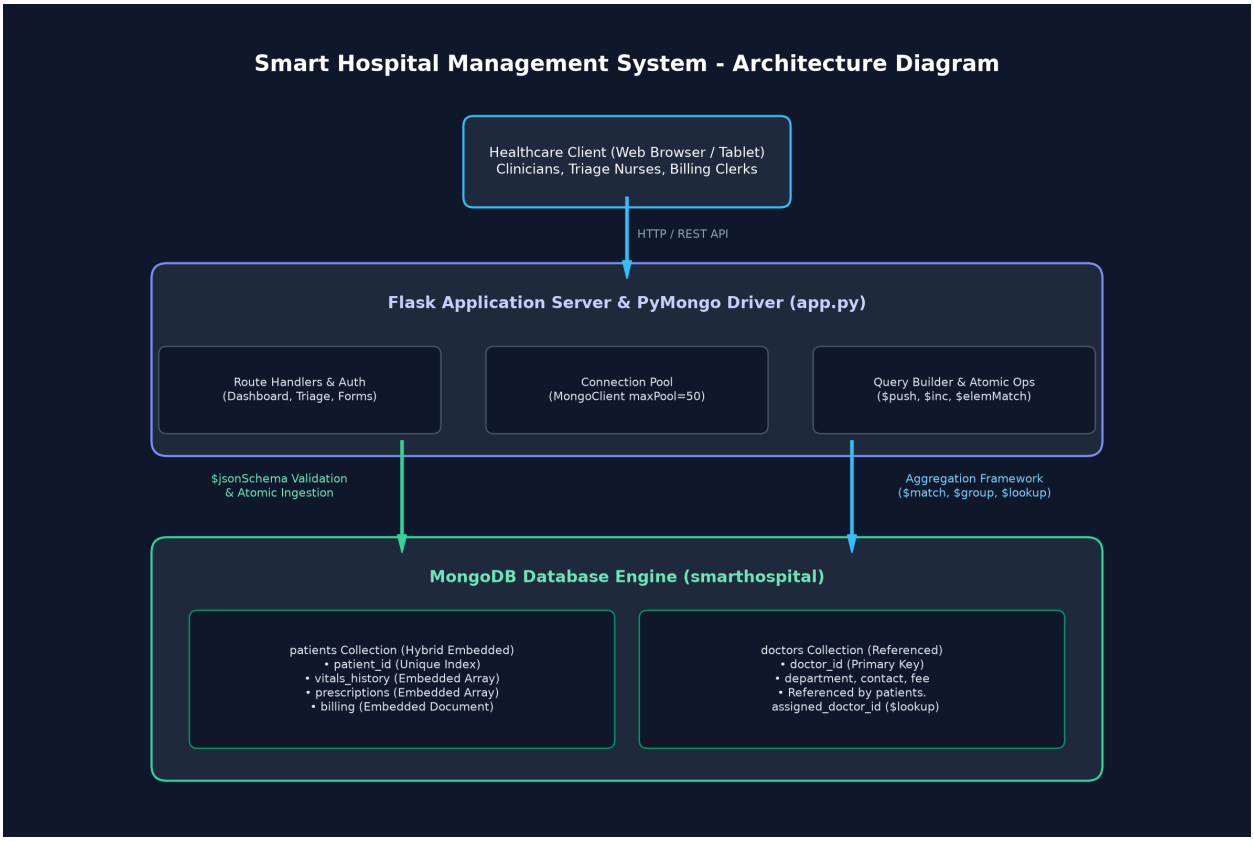


Figure 1: Multi-Tier System Architecture (Flask Client, PyMongo Driver, MongoDB Engine)

5.2. Application Screenshots: Ward & Patient Management

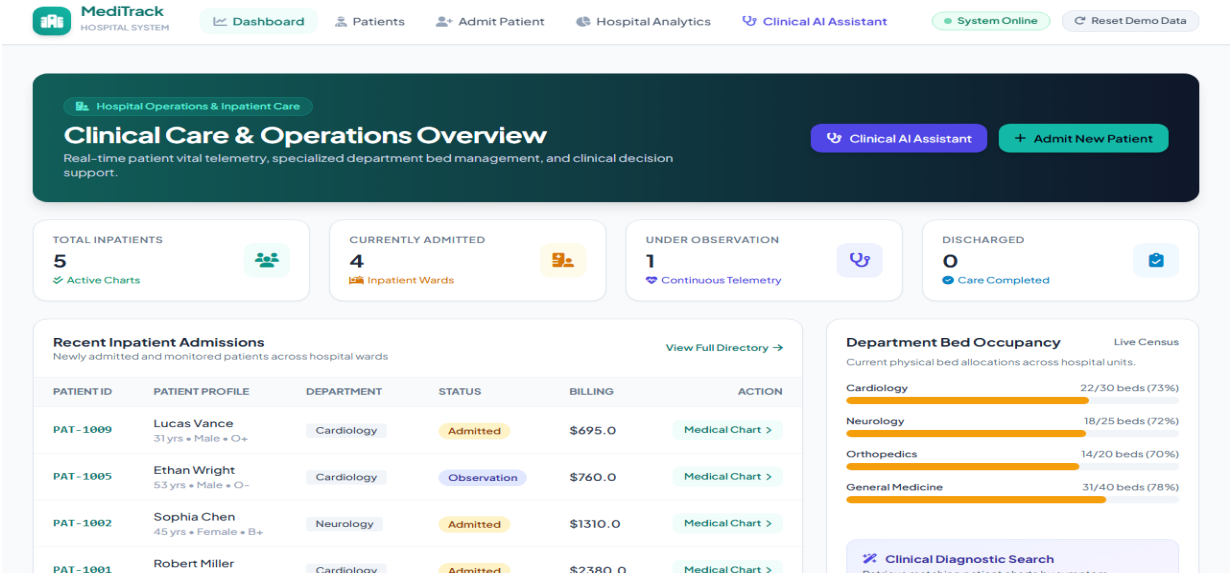


Figure 2: Executive Hospital Dashboard (Census, Real-Time Triage, Navigation)

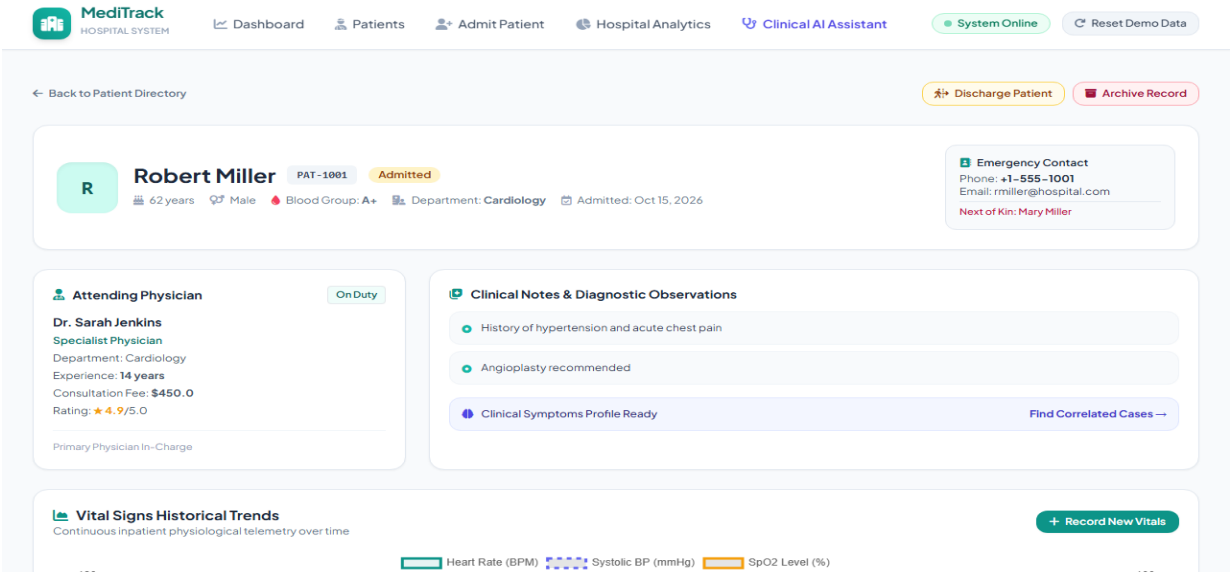


Figure 3: Inpatient Medical Chart (Embedded Vitals Telemetry, Prescriptions, Ledger)

5.3. Application Screenshots: Analytics & AI Clinical Assistant

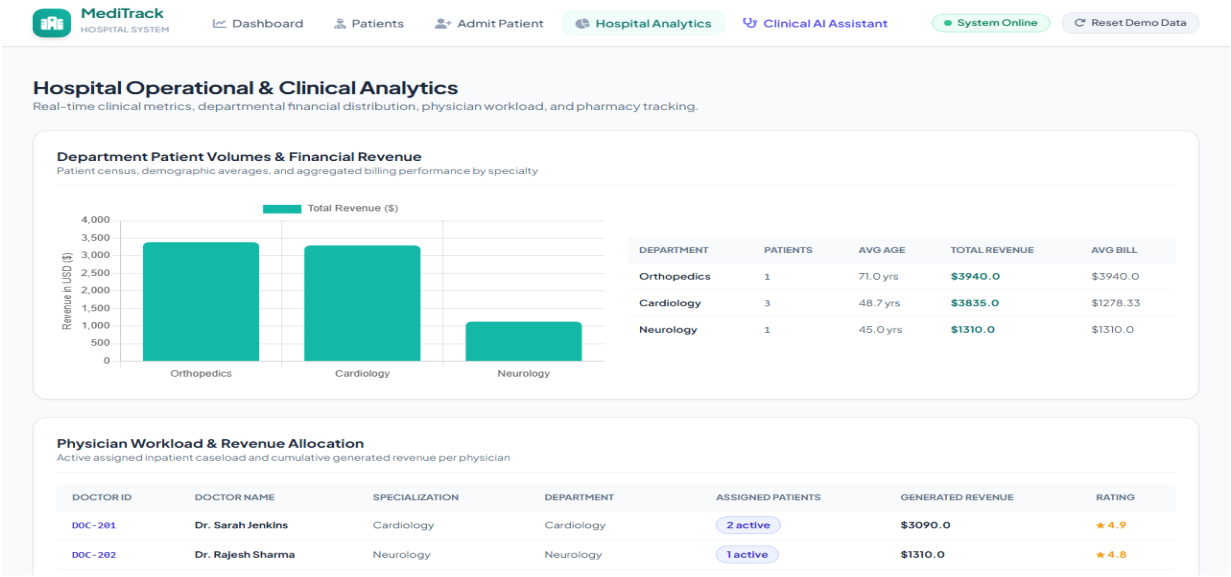


Figure 4: Clinical Analytics Dashboard (Aggregation Pipelines: Revenue, Caseload, Alerts)

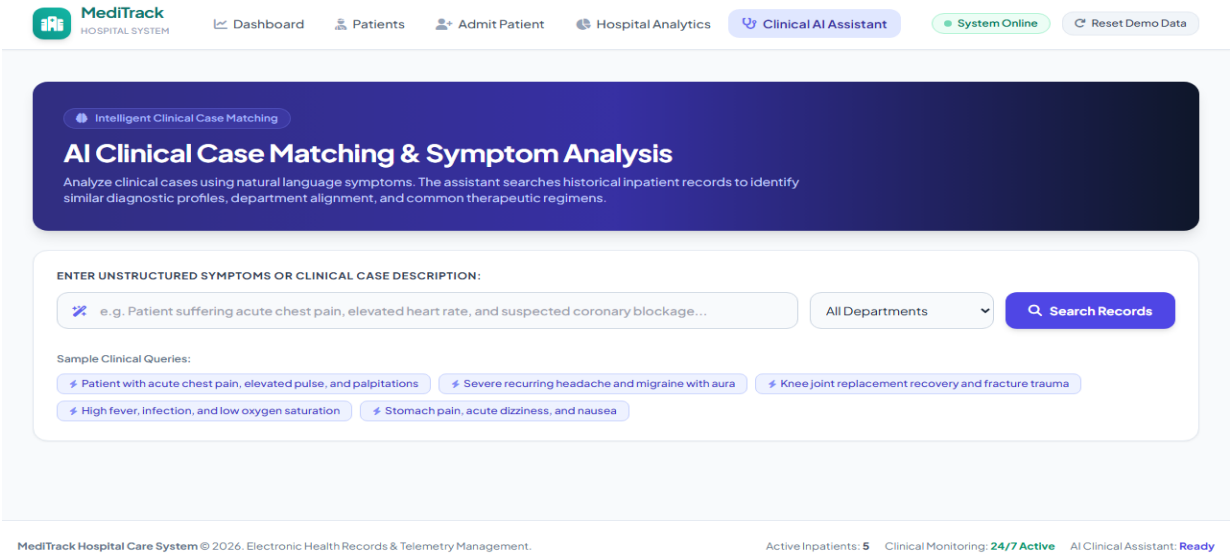


Figure 5: AI Clinical Assistant (Vector Similarity Search across Patient Diagnostic Notes)